

PNGres

A Primate-Addressable Persistence Layer

Technical Specification · Version 0.1 · 2026-04-23

Abstract. PNGres is a local-first data store in which every stored byte is represented as a low-order bit of a 650×800 RGBA image of a monke. Arbitrary binary inputs are sharded across a sequence of PNG files that are visually indistinguishable from the cover image. Payload data is embedded in the least-significant bits of

the pixel stream; structural metadata is carried in private PNG ancillary chunks. The system supports byte-accurate random access, which permits HTTP range requests and, consequently, streaming playback of MP4 video directly out of the resulting herd. This document specifies the on-disk format, the HTTP API surface, and observed performance characteristics.

Contents

Motivation 4
Architecture Overview 5
On-Disk Format 6
Overall Shard Structure 6
`mkHd` — Fixed 64-Byte Header 6
`mkMf` — Manifest JSON 6
`mkCk` — Per-Shard Integrity Hash 7
LSB Embedding Layout 7
Capacity 7
HTTP API 8
Range Request Handling 8
P2P: Connecting to a Remote Herd 8
Performance 10
Storage Footprint 10
Testing Summary 11
Limitations and Future Work 12
Glossary 13
References 14

Motivation

The author was asked, verbatim, for "a database tool that ONLY stores data for any file type or any type of data in headers and data of the monke.png file thats in this dir." The request further stipulated that the resulting system be funny, fully functional, streaming-capable, and scalable. PNGres is the response.

We frame the monke as a *content-addressable storage primitive*. Each shard is a self-describing unit of approximately 508 KiB of payload (at the default 2-bits-per-channel density), carrying a complete copy of the file-level manifest in a private ancillary chunk. This allows the system to tolerate most operational hazards — partial loss, metadata stripping, or filesystem reordering — while still presenting the user with what they asked for, which is a directory of monkes.

Architecture Overview

The implementation is a Rust Cargo workspace of three crates plus a static frontend bundle:

Crate	Responsibility
`pngres-core`	Pure encoding/decoding library. PNG chunk I/O, LSB embed/extract, manifest (de)serialization, streaming random-access reader.
`pngres-server`	HTTP frontend on `axum 0.7` . Routes, SQLite metadata (via `sqlx`), filesystem shard storage, embedded static assets (via `rust-embed`).
`pngres-cli`	Command-line interface on `clap 4` . Directory and zip I/O. Feature-gated `serve` subcommand.
`web/dist/`	HTML + CSS + vanilla JS frontend. No Node.js toolchain.

Request flow for a Range-streamed MP4:

```
[browser] --GET /api/files/:id/stream  Range: bytes=A-B-->
[server]  --SELECT manifest_json FROM files WHERE id=?-->
[server]  --compute first=A/per, last=B/per-->
[server]  --open only shards [first..=last]-->
[core]    --decode_png + LSB extract for each shard-->
[server]  --Body::from_stream(Stream<Result<Bytes>>)--
[browser] <--206 Partial Content, Content-Range: A-B/total--
```

On-Disk Format

Overall Shard Structure

A shard is a valid PNG file. Chunks appear in the following order:

Chunk	Purpose
`IHDR`	Standard PNG header. 650×800, 8-bit RGBA, non-interlaced.
`mkHd`	PNGres fixed 64-byte header (see below).
`mkMf`	Full file manifest as JSON. Duplicated in every shard.
`IDAT`	Pixel data with payload LSB-embedded. Always present.
`mkCk`	32-byte BLAKE3 hash of this shard's payload.
`IEND`	Standard PNG terminator.

The `mk*` chunks use the PNG convention for *ancillary, private, safe-to-copy* chunks (lowercase letters at positions 1, 2, and 4; uppercase at position 3). Viewers ignore them; copy-safe means routine PNG tools preserve them across re-saves.

`mkHd` — Fixed 64-Byte Header

Off	Size	Field	Notes
`0`	`8`	magic	ASCII `"PNGRES01"` .
`8`	`1`	format_version	`0x01` .
`9`	`1`	lsb_bits	1, 2, or 4.
`10`	`2`	flags	bit 0 reserved for payload compression.
`12`	`16`	file_id	UUIDv7 raw bytes.
`28`	`4`	chunk_index	u32 big-endian, 0-based.
`32`	`4`	total_chunks	u32 big-endian.
`36`	`8`	byte_offset	u64 BE, payload's offset in original file.
`44`	`4`	payload_length	u32 BE, real bytes in this shard.
`48`	`8`	total_file_size	u64 BE.
`56`	`4`	header_crc32	CRC32(bytes 0..52).
`60`	`4`	reserved	zero.

`mkMf` — Manifest JSON

Serialized form of the `Manifest` struct, `serde_json` -encoded:

```
{
  "file_id": "019dbe71-4699-72f0-90df-f6d37d8fe0cd",
  "filename": "movie.mp4",
  "mime": "video/mp4",
  "total_size": 106290,
  "num_monkes": 1,
  "lsb_bits": 2,
  "payload_per_monke": 519904,
  "file_hash": [/* 32 bytes */],
  "shard_hashes": [[/* 32 bytes */]],
  "created_at": 1777016587,
  "format_version": 1
}
```

`mkCk` — Per-Shard Integrity Hash

32 bytes. BLAKE3 of the shard's payload bytes (not of the whole PNG).

LSB Embedding Layout

The payload is embedded into the low `lsb_bits` bits of each pixel byte, in document order (R, G, B, A, R, G, B, A, …). The LSB stream carries:

```
| 64 B mkHd mirror | payload_length bytes of data | 32 B BLAKE3 |
```

The `mkHd` mirror is redundant with the ancillary chunk, deliberately. If a PNG re-saver strips the `mk*` chunks — some do, despite the safe-to-copy bit — the decoder falls back to LSB extraction. Every bit depth is attempted; whichever yields a valid magic + CRC wins.

Capacity

$650 \times 800 \times 4 = 2{,}080{,}000$ channel bytes. Capacity and overhead:

LSB bits	raw capacity	user payload (after 96 B overhead)	visible?
1	260,000 B	253.8 KiB	invisible
2	**520,000 B**	**507.7 KiB**	**almost invisible (default)**
4	1,040,000 B	1,015.5 KiB	banding visible on smooth regions

HTTP API

All routes return JSON unless otherwise noted. Errors return `{ "error": { "code": <string>, "message": <string>, "hint": <string> } }` ; the `hint` carries an informative and non-serious remark.

Method	Path	Purpose
`GET`	`/healthz`	Plain text `"ok"` .
`GET`	`/api/stats`	`{ total_files, total_bytes, total_monkes, oldest, newest }` .
`POST`	`/api/upload`	`multipart/form-data` ; field `file` . Returns `{ id, name, mime, size, num_monkes }` . 2 GiB cap.
`GET`	`/api/files`	Paginated list (`?limit=&offset=`), newest-first.
`GET`	`/api/files/:id`	Full metadata incl. the `Manifest` .
`GET`	`/api/files/:id/download`	Full body, `Content-Disposition: attachment` .
`GET`	`/api/files/:id/stream`	Range-aware. 206 on Range headers, 200 otherwise. `Accept-Ranges: bytes` .
`GET`	`/api/files/:id/monkes`	P2P: list the raw shard PNGs (index + per-shard URL). No pixel decode server-side.
`GET`	`/api/files/:id/monkes/:n`	P2P: serve the n-th raw monke. `Content-Type: image/png` , immutable cache.
`DELETE`	`/api/files/:id`	Soft delete. 204. Shard directory is removed.
`GET`	`/` , `/static/{*path}`	Embedded static frontend.

Range Request Handling

Given a header `Range: bytes=A-B` , the server computes the covering shard window as `first = A / per_monke` , `last = B / per_monke` , then opens only those shards. Out-of-bounds ranges yield 416. Suffix ranges (`bytes=-N`) and open-ended ranges are capped at 8 MiB so browsers issue follow-up Range requests rather than streaming the whole file. Multi-range is rejected (not needed for `<video>` ; common browsers never send it for media).

P2P: Connecting to a Remote Herd

PNGres instances can pull monkes from each other. The protocol is plain HTTP: one server serves its shards at `/api/files/:id/monkes/:n` , and any client (`pngres-cli` , another server, or curl) fetches them by URL.

The `pngres remote` CLI group wires this up:

```
pngres remote list <URL>
pngres remote discover <URL> <FILE_ID>
pngres remote pull <URL> <FILE_ID> --out <FILE>
pngres remote pull-monkes <URL> <FILE_ID> --out <DIR>
pngres remote mirror <URL> --out <DIR>
```

- `list` — prints the remote's file table.
- `discover` — fetches `/monkes` and reports the shard index (file_id, filename, per-shard URL, blake3 of the whole file).
- `pull` — uses `/stream` for an already-decoded bytestream, verified against the manifest's BLAKE3.
- `pull-monkes` — fetches the raw PNG shards one by one, validates each decodes locally and carries the expected `file_id` + `chunk_index` . The resulting directory can be handed to `pngres decode` to rebuild the original file without ever talking to the source server again.
- `mirror` — pulls every live file from a remote into `<DIR>/<file_id>/` .

A shard served over `/monkes/:n` carries `Cache-Control: public, max-age=31536000, immutable` , since the payload is content-addressed by the `mkCk` BLAKE3. CDNs, local caches, and P2P fan-out strategies are all free to dedupe by URL without risk.

The reference flow for "pulling a file off a remote and converting to real data" is:

```
GET /api/files/:id/monkes           -> { num_monkes, monkes: [url, ...] }
for each m in monkes:
  GET /api/files/:id/monkes/:n       -> image/png
  pngres_core::decode_shard_bytes(&png)  // locally, no server help
  append shard.payload to output
assert blake3(output) == manifest.file_hash
```

No trust in the remote server for payload integrity is required: every shard carries its own BLAKE3 in the `mkCk` chunk (and mirrored in its LSB footer), and the complete file's BLAKE3 lives in every shard's `mkMf` manifest.

Performance

Measured on Apple silicon (arm64, 8-core, `cargo 1.95` , `rustc 1.95` , release profile with `lto = "thin"`). Bench harness: `criterion 0.5` .

Operation (2 LSB bits)	**1 MiB input**	**10 MiB input**
Encode (parallel via `rayon`)	182 MiB/s	473 MiB/s
Decode (sequential, single core)	64 MiB/s	72 MiB/s

PNG re-compression at `Compression::Fast` dominates encode time. Decode is IDAT inflation plus LSB extraction; we deliberately keep it sequential so HTTP range consumers see strictly ordered bytes without reorder buffers.

Storage Footprint

At 2-bit LSB, one monke on disk weighs 700 KiB after PNG compression (much of which is determined by the cover's natural entropy plus the LSB churn). Per GiB of user payload, expect roughly:

$$2065 \text{ monkes} \approx 1.4 \text{ GiB of disk}$$

a storage blow-up factor of 1.4×. The LSB changes are high-frequency and compress poorly, which is why the factor exceeds 1.

Testing Summary

Green as of this document's timestamp:

Count	Coverage
21	Core unit tests (`capacity` , `chunk` , `cover` , `encode` , `hashing` , `lsb` , `manifest`).
14	Real-fixture roundtrips (txt/rs/json/pdf/png/jpg/mp3/mp4/empty/1-byte, MP4 at 1/2/4 LSB, 25 random byte-ranges into MP4).
11	Synthetic-payload roundtrips incl. boundary and cross-boundary cases.
3	Random-access tests (crossing, zero-length, past-EOF, missing-shard).
3	Property tests (`proptest`) — arbitrary `Vec<u8>` roundtrips at each bit depth.
1	`mkHd` strip → LSB-mirror fallback.
5	CLI happy-path via `assert_cmd` incl. dir+zip roundtrip and the exit-code-6 dest-exists case.
3	P2P API: full MP4 flow, multi-fixture stats, pull-then-redecode preserves bytes across 5 file types.

Total: 61 tests green across 8 test binaries.

End-to-end streaming is verified against a real H.264 MP4 fixture: the server is booted, the file uploaded, HTTP range requests issued, responses diffed against the original byte range via `cmp` , and the reconstructed file is validated by `ffprobe` to be a structurally-intact H.264 stream.

Limitations and Future Work

- **No encryption.** LSB steganography is obfuscation, not secrecy. Encrypt inputs before handing them to PNGres if confidentiality matters.
- **No deduplication.** Identical uploads produce disjoint herds. Trivial to add on the back of the BLAKE3 index (`idx_files_blake3`).
- **Single-writer.** SQLite in WAL mode supports one writer. For higher concurrency, swap the pool for Postgres; the DDL is ANSI-clean.
- **In-memory upload buffer.** The v1 upload path reads the whole multipart field into memory before encoding. A streaming variant is feasible given the shard-parallel encoder.
- **Thumbnailing.** A natural extension is to decode the first monke of any uploaded image/video and extract a `<video>` -poster frame eagerly. Not implemented.
- **Object storage.** `FsStore` is a trait-shaped primitive; an `object_store` S3 impl that translates `read_range` into S3 byte-range `GET` s is a straightforward addition.

Glossary

---

monke The atomic unit of PNGres storage. One 650×800 RGBA PNG carrying up to 508 KiB of user payload at the default bit depth.

herd A collection of monkes forming the complete encoding of one file. Formally, the set $\{m_0, ..., m_{n-1}\}$ where n is the shard count.

LSB (Least-Significant Bit) The lowest-order bit of a byte. At `lsb_bits = 2` we use the two lowest bits of every channel byte.

ancillary chunk A PNG chunk whose first letter is lowercase, meaning decoders may ignore it. PNGres defines `mkHd` , `mkMf` , and `mkCk` .

manifest The file-level metadata record — filename, MIME, total size, shard count, bit depth, BLAKE3 hash of the whole file, per-shard hashes.

chunk_index The 0-based position of a shard within its herd. Equivalent to its payload's byte offset divided by the per-monke payload size.

payload_per_monke At a given bit depth, the number of user bytes that fit into a single shard after subtracting the 96-byte LSB overhead (64 B header mirror + 32 B payload hash).

References

---

1. **PNG Specification**, W3C Recommendation, RFC 2083. https://www.w3.org/TR/png/
2. **BLAKE3 Cryptographic Hash**, Aumasson, Neves, Wilcox-O'Hearn, Winnerlein, 2020. https://github.com/BLAKE3-team/BLAKE3-specs
3. **UUID Version 7**, IETF RFC 9562. https://datatracker.ietf.org/doc/rfc9562/
4. **HTTP/1.1 Range Requests**, IETF RFC 9110 §14. https://datatracker.ietf.org/doc/html/rfc9110\#section-14
5. **axum web framework**, tokio-rs. https://docs.rs/axum/
6. **sqlx**, launchbadge. https://docs.rs/sqlx/